What is a Git Branch?

A branch is a safe copy of your project — a parallel line of development that lets you make changes, try features, or fix bugs without touching the main production code. Think of a branch like a personal working copy: you can experiment freely, and only merge back when changes are stable.

Real-life analogy: imagine a Google Doc. The main document is the published final version. Creating a branch is like making a copy of that doc, editing the copy, and then pasting your improvements back into the main document when everything is correct.

The "Main" Branch

Every new Git repository has a default branch — usually named **main** (older projects may use `master`). This branch represents the live, production-ready code: what users see on your website or app.

The golden rule: never commit experimental or unfinished work directly to `main`. Always create a new branch for features, fixes, or experiments so the production code remains stable and deployable.

Creating and Switching Branches — Safe Development

Basic workflow to start working safely:

Check current branch

git branch — shows all branches. The one with the * is active.

Create a branch

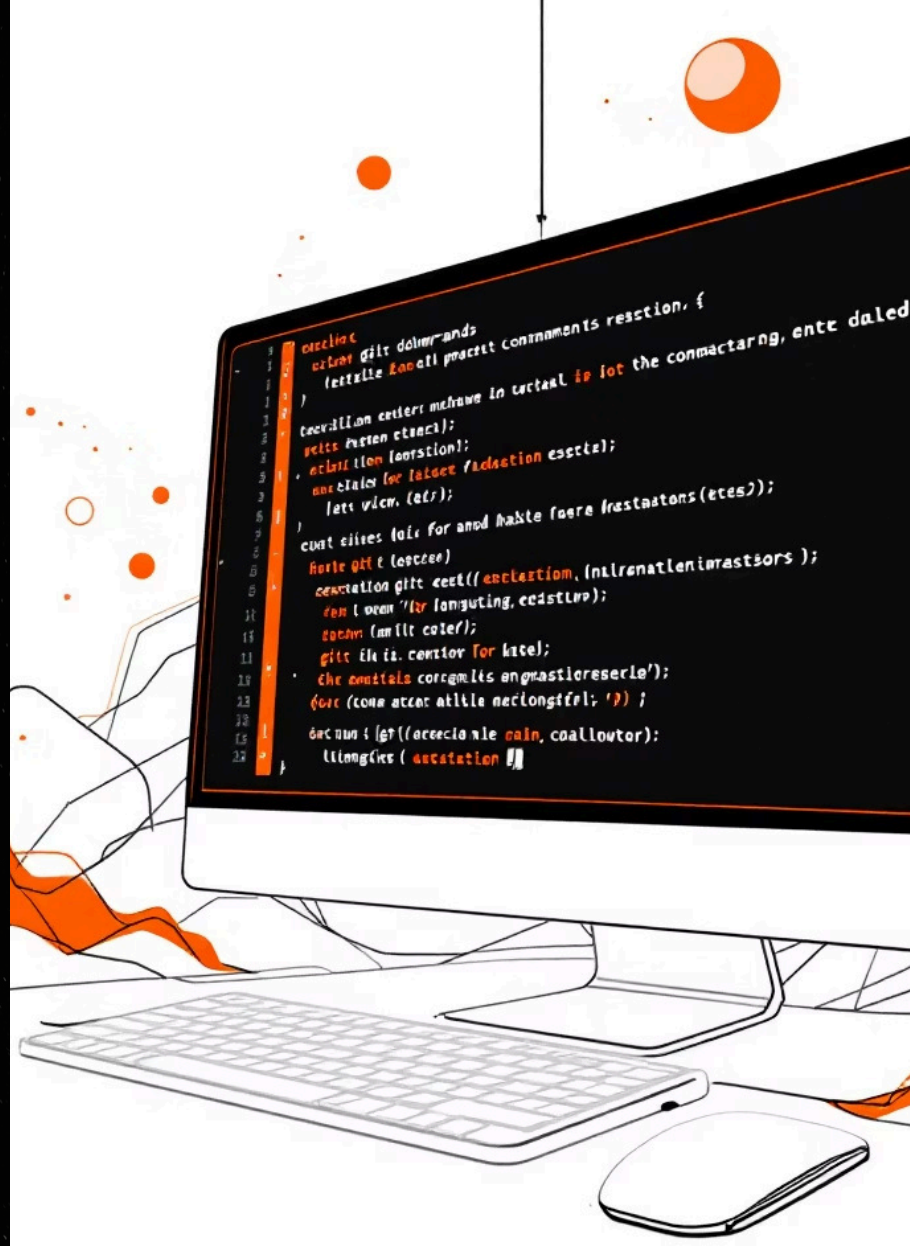
git branch feature-login — creates a branch named feature-login.

Switch branches

git checkout feature-login — move into that branch to begin work.

Pro tip (one step)

git checkout -b feature-login — create and switch in a single command.



Saving and Uploading Your Work — Daily Developer Routine

Once you're coding in your branch, follow these 3 steps to save and push your work to the remote repo so others can review it.

1. Add changes

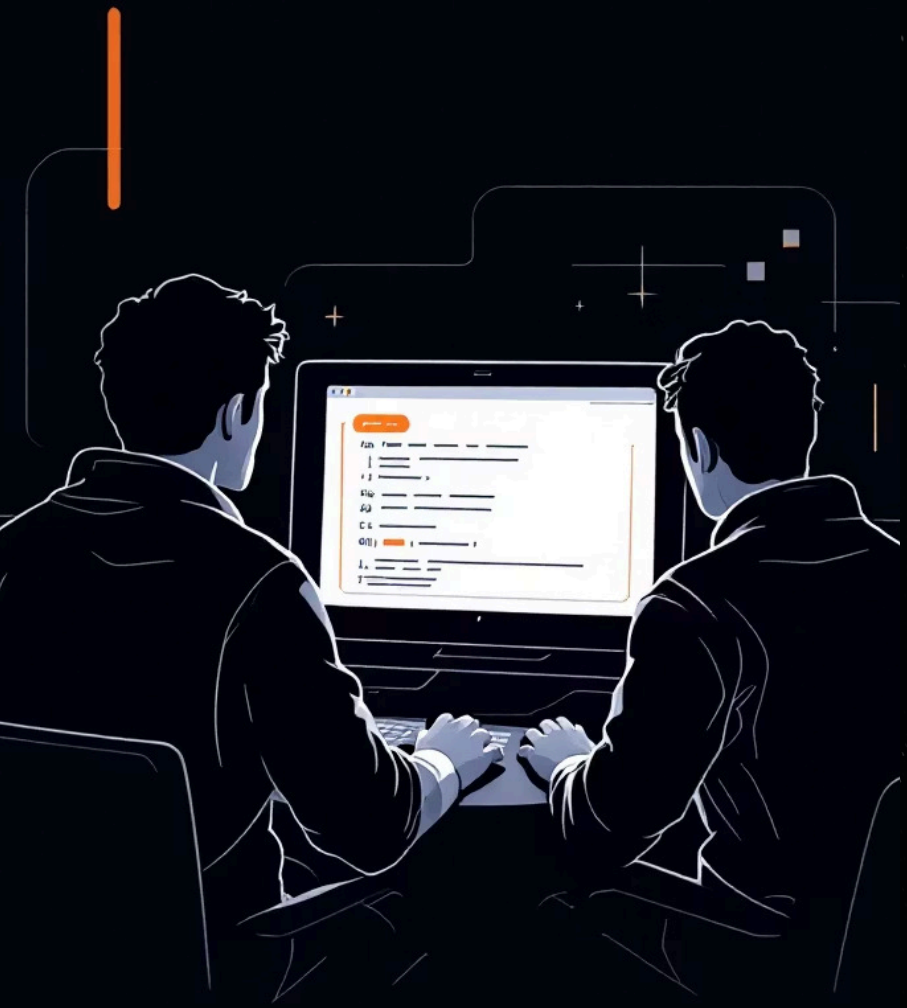
git add . — stage changed files you want to include in the commit.

2. Commit

git commit -m "Login page created by sarim" — save a snapshot with a clear message.

3. Push

git push origin feature-login — upload your branch to GitHub for sharing or PRs.

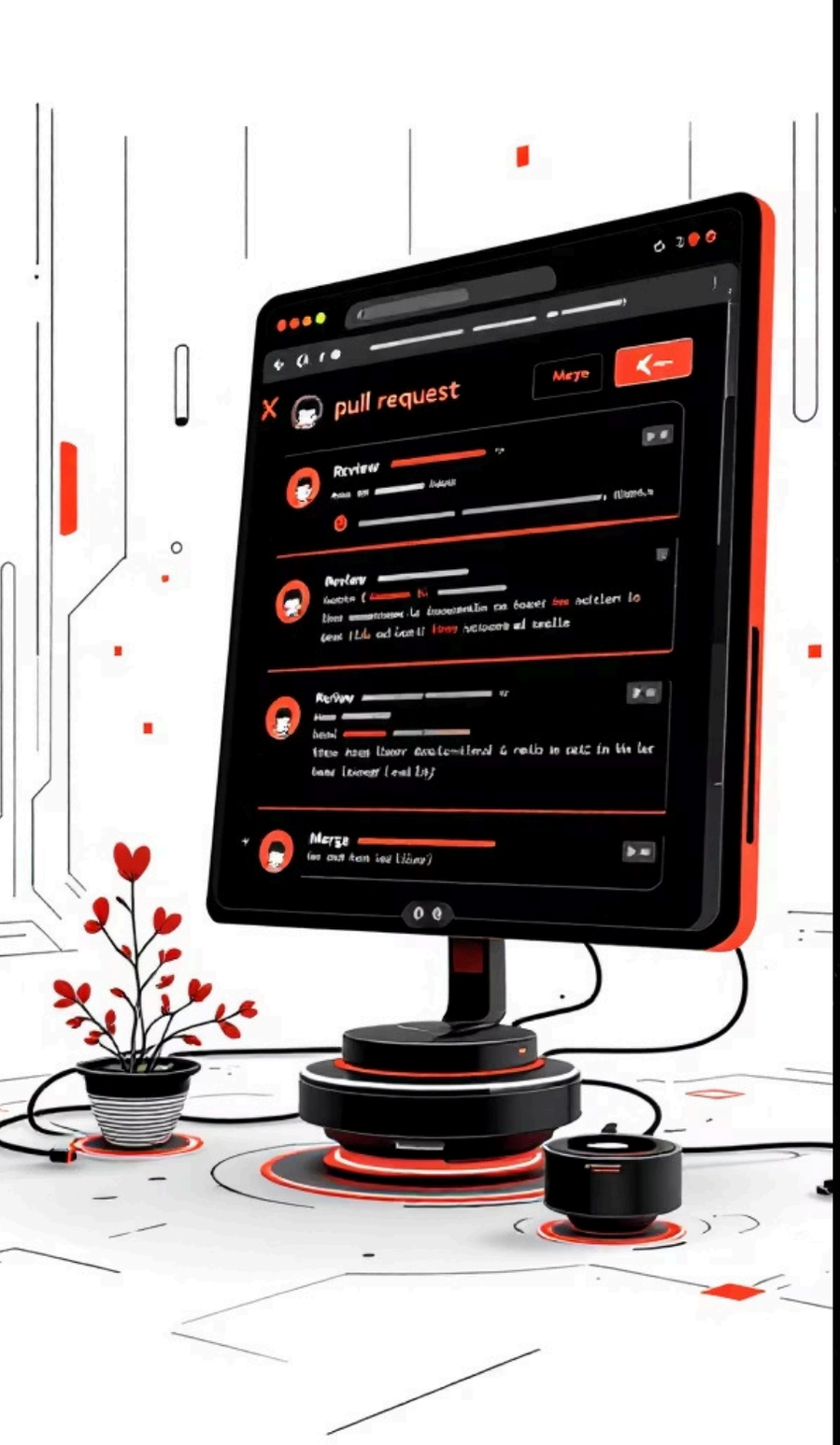


Merging Code Back to Main

When your feature is complete and tested locally, merge it into the live codebase carefully. Basic local merge steps:

1. git checkout main — switch to the production branch.
2. git pull origin main — update local main with remote changes.
3. git merge feature-login — combine your finished branch into main.

If merge conflicts appear, Git will pause and show the files to resolve. Edit the conflicts, stage the resolved files (git add .), and commit the merge to finish.



The Pull Request Workflow — How Teams Ship Safely

In professional teams you usually don’t merge locally — you open a Pull Request (PR) on GitHub so others can review your code before it reaches main.

01

1. Push your branch

git push origin feature-login — make your branch visible on GitHub.

02

2. Open a PR

On GitHub click "Compare & pull request", add a description, and request reviewers.

03

3. Code review & CI

Team members review, comment, and automated tests run (CI) — fix any issues if needed.

04

4. Merge

After approval and passing tests, click "Merge pull request" to bring changes into main.

Team Branching Strategy — Main, Develop, Features

Many teams use an intermediate `develop` branch for integration and testing before changes reach `main`. This reduces risk and groups features together for release.

Main

Production-ready code. Releases are built from `main`.

Develop

Integration/testing branch where multiple features are combined and validated.

Feature Branch

Personal workspaces (e.g., `feature-login`) for building one feature or fix.

Typical flow: Feature → Develop (test) → Main (release). This pipeline helps teams coordinate larger releases and run integration tests before deployment.

Cheat Sheet — Key Git Commands

git clone <url>	Download a project to your machine
git branch	List branches and see the current branch
git checkout -b <name>	Create a new branch and switch to it
git add .	Stage all changed files for commit
git commit -m "msg"	Save a snapshot with a clear message
git push origin <branch>	Upload your branch to the remote repository
git pull origin main	Fetch and integrate remote changes on main
git merge <branch>	Combine another branch into your current branch

Use descriptive commit messages and push frequently so teammates can review your work and CI can run tests.

Interview Prep — Common Question

Q: Why do we use branches in Git?

A: Branches allow developers to work independently on features or fixes without risking the stability of the main project. They enable parallel development, safer experimentation, easier code review, and clearer release workflows — all essential in team environments.

Practice explaining one workflow: feature → PR → review → merge. Real interviewers like concise, concrete examples.

Summary & Next Steps

Branches are your safety net: create feature branches, commit with clear messages, push for PR reviews, and merge only after testing. Use `develop` if your team needs a staging integration step. Remember the golden rule — keep `main` clean and production-ready.

Try this now

Make a small repo, create feature branch, implement a tiny change, and open a PR on GitHub to practice the full cycle.

Useful habit

Commit early, push often, and write clear commit messages to help reviewers and your future self.